Proje 1 — Rock Paper Scissors Vision

Exposure AI Academy · Winter 2027

Projenin Amacı

Bu projede elinle yaptığın taş, kağıt ve makas hareketlerini kameradan anlık olarak tanıyan bir yapay zekâ ürünü geliştireceksin. Hazır bir dataset kullanmayacaksın: kendi fotoğraflarınla kendi datasetini oluşturacak, biri pretrained, diğeri sıfırdan eğitilen iki modeli bu datasetle sırayla eğitip her birini kendi elinle deneyecek ve hangisinin daha iyi olduğuna F1 skoruna ve canlı teste bakarak karar vereceksin.

Kullanıcı:

- Kamerasını açacak
- Eliyle taş, kağıt veya makas yapacak
- Elinin etrafında bir kutu (bounding box) görecek
- Kutunun üstünde hareketin adını ve modelin ne kadar emin olduğunu (% confidence) görecek

Bu projede şunları öğreneceksin:

- Kendi datasetini toplamayı ve temizlemeyi
- Hazır eğitilmiş bir modeli kendi verinle yeniden eğitmeyi (post-training / fine-tuning)
- Pretraining'in ne kadar fark yarattığını, pretrained bir modeli sıfırdan eğitilen bir modelle karşılaştırarak görmeyi
- Jupyter Notebook kullanarak model eğitmeyi
- Performans metriklerine bakarak iki modeli karşılaştırmayı
- Kazanan modele odaklanıp diğerini bırakmayı (double down)
- Eğittiğin modeli canlı bir ürüne çevirip Vercel'e deploy etmeyi
- Bu sürecin her adımında Claude Code'u yönlendirmeyi

Görev Tanımı

Kendi datasetinle iki farklı CNN modelini eğit, en iyisini seç ve onu canlı bir ürüne dönüştür. Proje 6 aşamadan oluşur:

1. **Dataset toplama:** Elinle taş, kağıt ve makas yaparken fotoğraflarını çek.

2. **Data cleaning:** Kötü fotoğrafları ayıkla, eli kırp ve datayı train / validation / test olarak böl.
3. **Model seçimi:** Sıfırdan bir CNN ile pretrained ConvNeXt-Tiny arasından hangisiyle başlayacağını seç, ilkini kendi elinle denedikten sonra diğerini de eğitip karşılaştır. Claude CNN'in mimarisini önersin, sen onayla.
4. **Training ve karşılaştırma:** Claude'un hazırladığı Jupyter Notebook ile önce seçtiğin modeli eğit ve metriklerine bak. İkinci model, ilkini denedikten sonra sen isteyince eğitilir.
5. **Dene ve karar ver:** Claude'un açtığı deneme sayfasında modeli kendi elinle dene. Diğer modeli de eğitmek istediğinde Claude'a söyle; ikisi de eğitilince yan yana dene ve web'e hangisinin gideceğini seç.
6. **Productize:** Seçtiğin modelle son ürünü yap ve Vercel'e deploy et.

Sistem nasıl çalışacak?

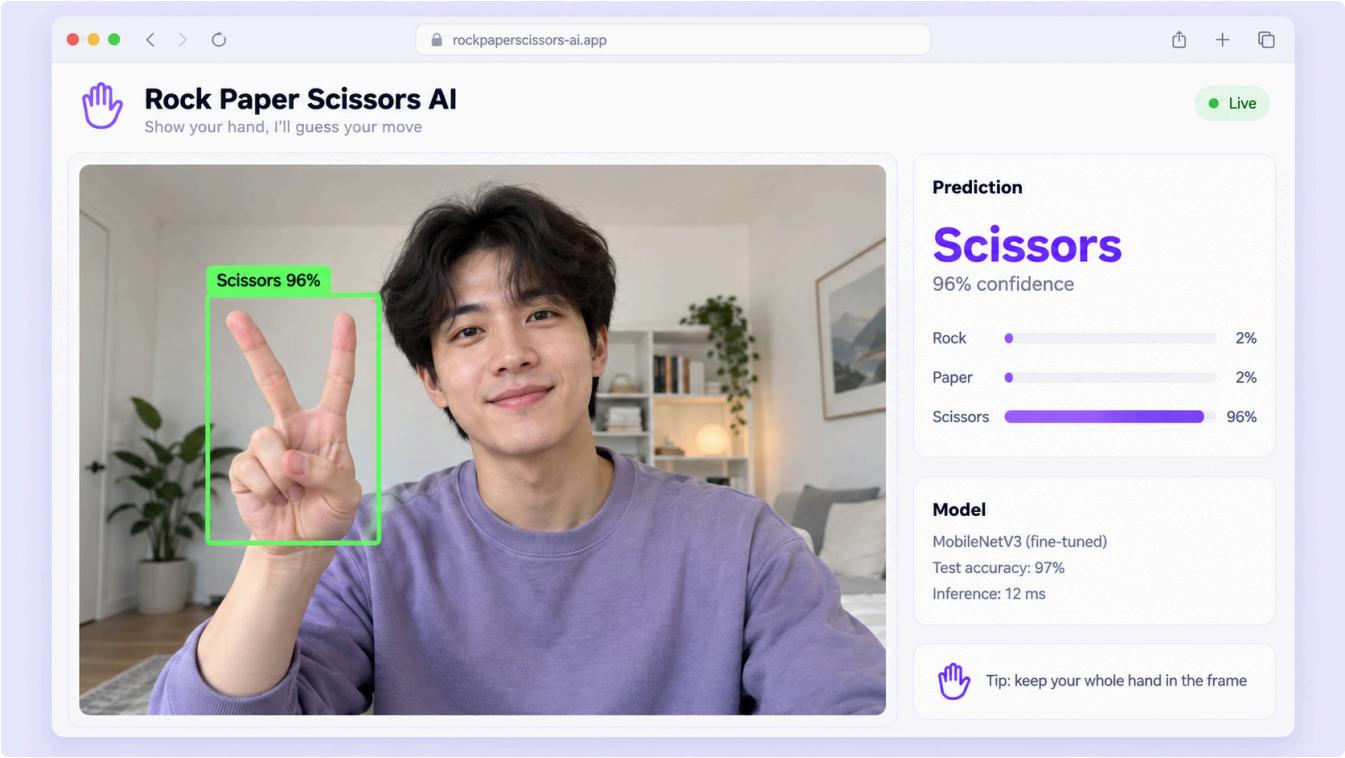
Kameradaki görüntüde iki iş yapılır:

- **Eli bulmak:** Hazır bir el tespit modeli (MediaPipe Hand Landmarker) elin nerede olduğunu bulur ve bounding box'ı çizer.
- **Hareketi tanımak:** Senin eğittiğin model, kutunun içindeki ele bakıp taş mı, kağıt mı, makas mı olduğuna ve yüzde kaç emin olduğuna karar verir.

Yani elin yerini hazır bir araç bulur, ama "bu hangi hareket?" sorusunu **senin eğittiğin model** cevaplar. Projenin asıl kısmı bu ikinci iştir.

Temel akış:

Fotoğrafları çek → Datayı temizle ve böl → Hangi modelle başlayacağını seç → Notebook'ta o modeli eğit → Kendi elinle dene → Diğer modeli eğit ve ikisini yan yana dene → Web'e gidecek modeli seç → Son ürünü yap ve Vercel'e deploy et



Aşama 1 — Dataset Toplama

Her hareket için en az 150, toplamda en az 450 fotoğraf çekeceksin. Modelin ne kadar iyi olacağını en çok bu aşama belirler: model sadece gördüğünü öğrenir.

Ortam ve ışık

- İyi aydınlatılmış bir yerde çek: gün ışığı veya tavan lambası yeterli.
- Işık kaynağı arkanda değil, önünde veya yanında olsun. Elin karanlıkta kalmasın.
- Sert gölgelerden kaçın.

Her hareket için çekim planı

Neyi değiştireceksin?	Nasıl?
Arka plan	3 farklı arka plan: düz duvar, masa üstü, karışık oda. Her birinde yaklaşık 50 fotoğraf.
Açı	Önden, sağa eğik, sola eğik, hafif yukarıdan, hafif aşağıdan, avuç içi ve el üstü.

Neyi değiştireceksin?	Nasıl?
Mesafe	Yakın (el kadrajın yarısı), orta, uzak (el kadrajın dörtte biri).
El	Hem sağ hem sol el.
Cihaz	Yarısını telefonla, yarısını laptop kamerasıyla çek. Demoda laptop kamerası kullanılacak.

Çekim kuralları

- Her fotoğrafta tek el ve tek hareket olsun.
- El tamamen kadrajda olsun. Parmaklar kesilmesin.
- Hareketi net yap: yarım açılmış makas veya yarım kapanmış kağıt olmasın.
- Bulanık fotoğraf çekme. El hareket ederken çekme.
- Aynı pozu art arda kopyalama. Her çekimde açıyı, mesafeyi veya eli biraz değiştir.
- Kol, yüz veya kıyafet görünebilir, sorun değil.

Klasör yapısı

Fotoğrafları şu şekilde düzenle:

```
dataset/
  rock/
  paper/
  scissors/
```

Fotoğrafları çektikten sonra `dataset/` klasörünü, Claude'un çalıştığı AWS'deki VS Code ortamına sürükleyip bırak. Data cleaning ve model eğitimi orada yapılacak.

İpucu: Claude'dan laptop kameranla tek tıkla fotoğraf çekip doğru klasöre kaydeden basit bir sayfa yapmasını isteyebilirsin.

Kontrol noktası: Claude her klasördeki fotoğraf sayısını göstersin. Üç klasörde de en az 150 fotoğraf olmalı ve sayılar birbirine yakın olmalı.

Aşama 2 — Data Cleaning

Temiz data, iyi modelin ön şartıdır. Bu aşamada Claude ile birlikte datayı modelin öğrenebileceği hale getireceksin.

Yapılacaklar:

1. **Göz kontrolü:** Her klasörü hızlıca gözden geçir. Bulanık, yanlış klasöre konmuş veya eli tam görünmeyen fotoğrafları sil.
2. **Eli kırpma:** Claude, MediaPipe ile her fotoğraftaki eli bulup sadece el bölgesini kırpan bir script yazsın. Demoda da model sadece kırpılmış eli göreceği için eğitimde de aynısını görmeli.
3. **El bulunamayanlar:** Elin bulunamadığı fotoğraflar ayrı bir klasöre ayrılsın. Bunlara bakıp silip silmeyeceğine sen karar ver.
4. **Boyutlandırma:** Tüm görseller aynı boyuta getirilsin.
5. **Bölme:** Data üçe bölünsün: yaklaşık %70 train, %15 validation, %15 test.

Altın kural: Test datası eğitimde asla kullanılmaz. Test datası, modelin daha önce hiç görmediği fotoğraflarda ne kadar başarılı olduğunu ölçmek içindir. Model test datasını görürse sonuçlar yalan söyler.

Kontrol noktası: Claude kırpılmış fotoğraflardan her sınıf için birkaç örneği bir grid olarak göstersin ve train / validation / test sayılarını tablo halinde versin. Kırpılmış görsellerde el ortada ve net görünmeli.

Aşama 3 — Model Seçimi

Bu aşamada iki seçeneğin var ve ikisini de deneyeceksin; hangisiyle başlayacağına sen karar vereceksin. İkisi aynı anda eğitilmez: önce seçtiğin model eğitilir, onu kendi elinle denedikten sonra diğerine geçersin. Seçeneklerden biri kendi CNN'ini sıfırdan eğitmek: ConvNeXt-Tiny'ye yakın büyüklükte (yaklaşık 25 milyon parametre) ve tamamen sana ait bir model. Diğer seçenek ise milyonlarca görselle önceden eğitilmiş (pretrained) ConvNeXt-Tiny'yi kendi datasetinle yeniden eğiteceksin; buna post-training veya fine-tuning denir. İki modelin sonuçlarını yan yana koyup aradaki farkı kendin göreceksin.

Seçenekler:

- **1. Seçenek — Model A: Kendi CNN'in (sıfırdan):** Claude'la birlikte tasarlayacağın bir CNN (örneğin 4–5 konvolüsyon bloğu, yaklaşık 25 milyon parametre). ConvNeXt-Tiny (yaklaşık 28 milyon) ile aynı büyüklükte olduğu için iki model arasındaki fark boyut değil, ön eğitim olur. Ağırlıkları rastgele başlar ve sadece senin datasetinle eğitilir. Eğitim başlayınca tarayıcıda canlı bir eğitim paneli açılır: modelin elinin fotoğrafında bulduklarını kulelerden bir şehir olarak görürsün ve Conv, BatchNorm, ReLU, MaxPool adımlarının bu şehre ne yaptığını izlersin.

- **2. Seçenek — Model B: ConvNeXt-Tiny (pretrained):** ImageNet üzerinde önceden eğitilmiş `convnext_tiny` (torchvision veya timm). Model A ile benzer büyüklükte; kendi datasetinle fine-tune edeceksin. Eğitim sürerken tarayıcıda Model A ile aynı kule şehri açılır ve farkları gösterir: fotoğraf önce karelere bölünür (Patchify), hazır beynin donuk kısımları gri, öğrenen kısımları mavi çizilir.
- **Adil karşılaştırma:** İki model aynı data, aynı augmentation ve aynı train / validation / test bölünmesiyle eğitilmeli. Aradaki asıl fark ön eğitim olmalı.
- **CPU'da eğitim:** Eğitim 48 vCPU çekirdekli, 96 GB RAM'li ama GPU'suz sunucumuzda yapılacak. ConvNeXt-Tiny CPU için ağır bir model: önce gövdeyi dondurup sadece sınıflandırma katmanını eğit, sonra son blokları açıp birkaç epoch daha fine-tune et. Giriş boyutunu 224 px tut ve tüm çekirdekleri kullan (PyTorch'ta `torch.set_num_threads(48)`, DataLoader'da `num_workers`).

Claude sana iki seçeneği anlatsın ve hangisiyle başlamak istediğini sorsun; karar senin. Claude sadece seçtiğin modeli eğitir, diğerini sen söyleyince eğitir. Sıfırdan CNN için yaklaşık 25 milyon parametreli bir mimari önersin, son mimariyi sen onayla. ConvNeXt-Tiny için de CPU'da hızlı bir fine-tuning planı çıkarsın.

Claude'a şunu sorabilirsin:

> Datasetimde her sınıf için yaklaşık X fotoğraf var. GPU'suz, 48 vCPU'lu, 96 GB RAM'li bir sunucuda iki model eğiteceğim: sıfırdan eğitilecek, yaklaşık 25 milyon parametreli bir CNN ve ImageNet'te pretrained ConvNeXt-Tiny. Bu dataset için uygun bir sıfırdan CNN mimarisi öner, ConvNeXt-Tiny'yi CPU'da hızlı fine-tune etmek için bir plan çıkar. Eğitim başlayınca açılacak canlı bir eğitim paneli hazırla; önceden çekilmiş video olmasın, eğitimin kendisinden çizilsin. Panel sade olsun, az yazı ve sayı içersin: Model A'da tek bir filtrenin elimin fotoğrafında bulduklarını kulelerden bir şehir olarak göster, Conv, BatchNorm, ReLU ve MaxPool adımlarının bu şehre ne yaptığı görünsün; Model B'de aynı şehirle farkları göster: Patchify, Conv 7×7, LayerNorm, GELU, Residual ve Downsample adımları görünsün; donuk kısımlar gri, öğrenen kısımlar mavi olsun. Panelleri baştan yazma: https://github.com/Darkosxl/exposure-academy-animations deposunun animations/ klasöründeki live.py ve live_cnn.html dosyalarını değiştirmeden kullan, README'deki gibi eğitim döngüsüne bağla. Önce seçtiğim modeli eğit; diğerini ben ilkini deneyip söyleyince eğit.

Kontrol noktası: İki modeli ve sıfırdan CNN için onayladığın mimariyi `DENEY_GUNLUGU.md` dosyasına yazdır. Hangisinin kazanacağına ve pretraining'in ne kadar fark yaratacağına dair kendi tahminini de ekle.

Aşama 4 — Training ve Metrik Karşılaştırma

Claude tüm eğitim akışını tek bir Jupyter Notebook (`training.ipynb`) olarak hazırlayacak. Sen de notebook'taki talimatları takip ederek hücreleri sırayla çalıştıracaksın. Notebook'ta her modelin kendi bölümü var: ortak hazırlık hücreleri ve sadece seçtiğin modelin bölümü çalıştırılır. Diğer modelin bölümü, sen ilk modeli deneyip istediğini söyleyene kadar çalıştırılmaz.

Notebook'ta olması gerekenler

1. Her hücrenin üstünde Türkçe kısa bir açıklama: bu hücre ne yapıyor ve çıktıda neye bakmalısın
2. Datasetten her sınıf için örnek görseller
3. Data augmentation (döndürme, parlaklık değiştirme vb.): modeli farklı koşullara hazırlamak için
4. Model A'nın (sıfırdan CNN) eğitimi; eğitim sürerken canlı eğitim paneli (aşağıda)
5. Model B'nin (ConvNeXt-Tiny) fine-tuning'i, aynı data ve aynı augmentation ile; eğitim sürerken canlı eğitim paneli (aşağıda)
6. Her model için eğitim grafiği: train ve validation F1 / loss
7. Test datasında sonuçlar: macro F1 ve sınıf başına F1
8. Her model için Grad-CAM görselleri: birkaç test fotoğrafının üstüne, modelin karar verirken elin hangi bölgesine baktığını gösteren ısı haritaları (sadece incelemek için)
9. İki modelin karşılaştırma tablosu (ikinci model de eğitildiğinde)
10. Her modelin, eğitimi biter bitmez ONNX formatında dışa aktarılması (deneme sayfası ve son ürün için)

Canlı eğitim paneli

Modeli seçip Claude eğitimi başlattığında, eğitim boyunca güncellenen canlı bir panel açılır. Panel önceden hazırlanmış bir video değildir: birkaç batch'te bir, modelin o anki haliyle yeniden çizilir. Sade ve renkli tutulur: az yazı, az sayı; kavramlar animasyonla görünür.

- **Model A (sıfırdan CNN):** Açık renkli, sade bir sayfa: üstte küçük bir "Model A öğreniyor" başlığı, ortada epoch sayacı. Her tur (yaklaşık 15 saniye) modelin o anki gerçek değerleriyle oynar. Eğitilen batch'in fotoğrafları şerit halinde kayar, biri çıkıp büyür. Conv (yavaş): tek bir damga (eğitim boyunca hep aynı filtre, gerçek 3×3 ağırlıklarından çizilir) fotoğrafın üstünden geçer; geçtiği her yer için eğik bir 3D şehirde

(8×8) bir kule yükselir. Kulenin yüksekliği, o yerin damgaya ne kadar benzediğidir. BatchNorm: bütün şehir aynı oranda kayıp esner, su çizgisi ortalamaya yerleşir. ReLU: suyun altında kalan her şey silinir. MaxPool: her 2×2 gruptan sadece en uzun kule kalır, şehir küçülür. Blok 2–5: aynı dört adım hızlıca tekrar eder ve şehir küçülmeye devam eder; her bloğun damgaları bir önceki bloğun şehrine bakar (çizgiler → şekiller → parmaklar → el). Tahmin: Taş, Kağıt ve Makas için üç çubuk; doğru cevap yeşil, yanlış tahmin kırmızı yanar. Her adımda sadece küçük adım adı (Conv, BatchNorm, ReLU, MaxPool) yanar; başka yazı, sayı veya emoji yoktur.

- **Model B (ConvNeXt-Tiny):** Model A ile aynı sayfa ve aynı kule şehri; farkları gösterir. Başlık "Model B öğreniyor", bir tur yaklaşık 17 saniye. Patchify: fotoğrafın üstüne bir ızgara çıkar; fotoğraf 4×4 piksellik, üst üste binmeyen karelere bölünür (gerçekte 56×56 kare, çizimde 16×16). Her kare tek seferde bir kule olur, şehir bir anda çıkar. Hiçbir piksel atlanmaz, atlanan şey Model A'daki üst üste binmedir: tek piksel tek başına bir şey anlatmaz, en küçük anlamlı parça 4×4 karedir ve sonraki katmanlar yaklaşık 16 kat az iş yapar. Sonra her aşamadan bir blok oynar (aşama 1 yavaş, 2–4 hızlı). Conv 7×7: daha büyük bir damga (gerçek 7×7 ağırlıklarından çizilir) geçer ve bir seferde daha geniş bir alana bakar. LayerNorm: su çizgisi ortalamaya gelir (BatchNorm ile aynı fikir). GELU: ReLU gibi, ama suyun altında kalanlar silinmez, kısa kütüklere ezilir. Residual: eski şehir altta açık renkte geri gelir, blok sadece üstüne bir parça ekler; bloklar üst üste dizilse de önceki bilgi kaybolmaz. Downsample (aşamalar arasında): her 2×2 karışıp tek kule olur (MaxPool sadece en uzunu tutardı), şehir küçülür. Donuk aşamalar gri, öğrenen aşamalar mavi çizilir: ilk adımda sadece karar kısmı mavidir, ince ayar adımı başlayınca son aşama da maviye döner. Altta her blok için küçük bir nokta (3, 3, 9, 3 ve karar) aynı renkleri gösterir; o an oynayan aşamanın noktaları büyür. Takip edilen damga, fotoğrafın ortasına (ele) en çok tepki veren kanaldır; hazır beyin sayesinde şehir ilk epoch'tan el şeklindedir. Tahmin: şehir karar kısmının renginde tek kuleye iner, sonra Model A'daki gibi Taş, Kağıt ve Makas çubukları yanar. Sadece küçük adım adları (Patchify, Conv 7×7, LayerNorm, GELU, Residual, Downsample) yanar; başka yazı, sayı veya emoji yoktur.
- **Panellerin kodu:** Panelleri baştan yazma. İkisinin kodu herkese açık [exposure-academy-animations] deposunun `animations/` klasöründe. `live.py` ve `live_cnn.html` dosyalarını değiştirmeden notebook'un yanına kopyala ve README'deki gibi eğitim döngüsüne bağla; böylece herkeste aynı animasyon çıkar.
- **Nerede açılır:** İki panel de eğitim başlayınca tarayıcıda kendiliğinden açılır: eğitimin içinde çalışan küçük bir yerel sayfadır. Uzak sunucuda VS Code bu sayfayı senin bilgisayarının tarayıcısında açar; açılmazsa link eğitim hücresinin çıktısında yazar.

İki sayfa da kendi hızında akıcı oynar ve değerleri birkaç saniyede bir eğitimden yeniden alır. İkisi de önceden çekilmiş bir video değil, eğitimin o anki gerçek durumudur.

Hangi metriklere bakacaksın?

Metrik	Ne anlatır?
Macro F1 (ana metrik)	Precision (model "makas" dediğinde ne sıklıkla haklı?) ile recall'u (gerçek makasların kaçını yakaladı?) tek sayıda birleştirir ve üç hareketi eşit ağırlıkla değerlendirir. İki modeli karşılaştırırken ilk buna bak.
Sınıf başına F1	Model bir harekette çok iyi, diğerinde kötü mü?

Dikkat etmen gereken bir uyarı işareti: train F1 çok yüksek ama validation F1 düşükse model fotoğrafları ezberlemiş demektir (overfitting). Bunu görürsen Claude'a sor ve ne yapabileceğinizi konuşun.

Bonus — Grad-CAM ile modelin içine bak: Grad-CAM, modelin bir fotoğrafa karar verirken en çok nereye baktığını ısı haritası olarak gösterir: kırmızı bölgeler kararda en etkili yerlerdir. İyi bir model parmaklara ve elin şekline bakar; arka plana veya kola bakıyorsa yanlış şeyi öğrenmiş olabilir. İkinci model de eğitildiğinde iki modelin ısı haritalarını yan yana karşılaştır. Bu kısım sadece merak ve inceleme için; kazananı F1 ve canlı teste bakarak seçeceksin.

Kontrol noktası: Eğittiğin modelin metriklerini (özellikle macro F1) `DENEY_GUNLUGU.md` dosyasına ekle. İkinci model de eğitildiğinde iki modeli karşılaştırma tablosu olarak yaz; rakamlara göre önde olan modeli ve Aşama 3'teki tahmininin doğru çıkıp çıkmadığını ekle.

Aşama 5 — Dene ve Karar Ver

Rakamlar bir şey söyler, ama asıl test gerçek hayattır. Model eğitilir eğitilmez Claude bir **deneme sayfası** açar: kameranı açıp modeli kendi elinle denersin. Sayfa VS Code'un yönlendirdiği bir localhost linkinde açılır; model sunucunun CPU'sunda çalışır.

Deneme sayfası:

- Laptop kamerana erişir
- Elini MediaPipe ile bulur ve eğitimdekiyle aynı şekilde kırpar
- Kırpılmış eli o ana kadar eğittiğin model(ler)e gönderir

- Her model için ayrı bir panelde elin etrafına bounding box, kutunun üstünde tahmin ve confidence (örneğin "Taş %94") ve üç hareketin olasılık çubuklarını gösterir
- Elin görünmüyorsa **Elini kameraya göster**, kamera açılmazsa **Kameraya erişilemedi. Lütfen izin ver.** yazar

Akış

1. İlk modelin eğitilince Claude deneme sayfasını açar. Farklı açılar, mesafeler ve ışıklarla dene; model nerede zorlanıyor, bak.
2. Diğer modeli de denemek istediğinde Claude'a söyle. Claude ikinci modeli sen söylemeden eğitmez ve iki modeli asla aynı anda eğitmez.
3. İkinci model eğitilince Claude deneme sayfasını yeniden açar; bu sefer iki model yan yana durur. Hangisinin daha emin ve daha tutarlı olduğuna bak.
4. Web'e hangi modelin gideceğine sen karar ver ve Claude'a söyle.

Claude'a şunu söyleyebilirsin:

> Model A'nın eğitimi bitti. Deneme sayfasını aç, kendi elimle deneyeyim.

Kontrol noktası: Kararını `DENEY_GUNLUGU.md` dosyasına yaz: hangi modeli seçtin, metrikler ve canlı deneme aynı şeyi mi söyledi, diğer model nerede zorlandı?

Aşama 6 — Productize ve Deploy

Kazanana double down et, kaybedeni kill et. Bu aşamada sadece kazanan modeli kullanan, herkesin açıp deneyebileceği temiz bir ürün yapacaksın. Claude ile bir web uygulaması (Next.js) yapacaksın; model tarayıcının içinde çalışır (ONNX Runtime Web), sunucu veya API anahtarı gerekmez.

Yapılacaklar:

1. Seçmediğin modeli projeden kaldır: model dosyası ve ilgili kodlar
2. Arayüzde tek model olsun: büyük kamera görüntüsü, bounding box, tahmin ve % confidence
3. Ürüne bir isim ve kısa bir açıklama ver
4. Telefonda da çalıştığını kontrol et (ön kamera)
5. GitHub'a push et
6. Vercel'e deploy et ve canlı linki telefonundan aç

Arayüzde olması gerekenler

1. Başlık ve kısa açıklama: örneğin **Taş Kağıt Makas AI — Elini göster, ne yaptığını tahmin edeyim.**
2. Canlı kamera görüntüsü
3. Bounding box, tahmin ve % confidence
4. Loading, hata ve "elini göster" durumları
5. Kullanılan modelin adı ve macro F1 skoru (küçük bir bilgi kartı)
6. Mobil uyumlu tasarım

Teknik Gereksinimler

Bu proje:

- Yeni bir repo olarak hazırlanmalı
- Claude Code ile, aşama aşama geliştirilmeli
- Kendi çektiğin fotoğraflardan oluşan bir dataset kullanmalı
- Model eğitimini Jupyter Notebook (AWS'deki VS Code notebook ortamında) ile yapmalı
- Sıfırdan eğitilen bir CNN'i, fine-tune edilmiş pretrained ConvNeXt-Tiny ile karşılaştırmalı
- Son üründe modeli tarayıcıda (ONNX Runtime Web) çalıştırmalı
- Localhost'ta çalışmalı
- GitHub'a push edilmeli
- Vercel ile deploy edilmeli
- Masaüstü ve mobilde düzgün çalışmalı

Git ve Deploy

Projeyi yeni bir repo olarak başlat ve her aşamanın sonunda commit at:

```
git add .
git commit -m "asama 1: dataset toplandi"
git push origin main
```

Repoda olması gerekenler: `training.ipynb`, `DENEY_GUNLUGU.md`, model dosyaları (`.onnx`) ve web uygulaması.

Repoda olmaması gerekenler: ham fotoğraflar. `dataset/` klasörünü `.gitignore` dosyasına ekle. Fotoğrafların hem büyük hem kişisel; onları Google Drive'da sakla.

Bu projede API anahtarı yok. Modeller tarayıcıda çalıştığı için Vercel'e environment variable eklemen gerekmez.

Zorunlu Gereksinimler

Tamamlanmış projede şunlar bulunmalıdır:

1. Yeni bir repo oluşturulmuş olmalı
2. Kendi çektiğin fotoğraflardan oluşan, her sınıfta en az 150 fotoğraf içeren bir dataset olmalı
3. Dataset temizlenmiş ve eller kırpılmış olmalı
4. Data train / validation / test olarak bölünmüş olmalı
5. Test datası eğitimde kullanılmamış olmalı
6. Model A sıfırdan eğitilen (pretrained olmayan) bir CNN, Model B pretrained ConvNeXt-Tiny olmalı
7. İki model de Jupyter Notebook ile, sırayla eğitilmiş olmalı: ikinci model, ilki kamerada denendikten sonra
8. Notebook'taki her adımın Türkçe açıklaması olmalı
9. Her model için macro F1 ve sınıf başına F1 gösterilmeli
10. İki modelin metrikleri bir tabloda karşılaştırılmalı
11. İki model ONNX formatında dışa aktarılmış olmalı
12. Deneme sayfası ve son ürün kameraya erişebilmeli
13. Deneme sayfası eğitilen modelleri yan yana, bounding box ve % confidence ile göstermeli
14. Loading, hata ve "elini göster" durumları gösterilmeli
15. Web'e gidecek model öğrenci tarafından seçilmiş ve kararın nedeni yazılmış olmalı
16. Son ürün sadece kazanan modeli kullanmalı
17. `DENEY_GUNLUGU.md` her aşamanın kararlarını içermeli
18. `dataset/` klasörü `.gitignore` içinde olmalı
19. Proje GitHub'a push edilmeli
20. Proje Vercel'e deploy edilmeli
21. Uygulama mobilde düzgün çalışmalı